



Exploring Code Conversion using AI

An experiment using generative AI to accelerate the process of converting ColdFusion to modern TypeScript and what we can learn from it.



With respect to AI, what is hype and what is real?

What do you do when you believe a new tool or technology can help, but you're blocked from using it?



A Little About Me...

I started my career in engineering and I still live vicariously through our engineers.

The last real production code I shipped in an enterprise environment was over 20 years ago.

I've never been a ColdFusion developer or a TypeScript developer.

You'll often find me saying, "Sure. Let's try it. What's the worst that could happen?"



The Situation

One of our customers has a portfolio of ColdFusion applications. They want to migrate to a more modern technology stack and we believe we can help, but...

We're not allowed to use generative AI technology within the client environment.

We can't get the existing code we want to convert outside the client environment.







TL;DR on the Experiment

We successfully converted a simple ColdFusion sample app to a functionally equivalent TypeScript app using agentic AI.

We established **better practices and process** for effective code conversions using AI.

AI dramatically accelerated the conversion *and* human expertise is still essential.

How Much Acceleration?

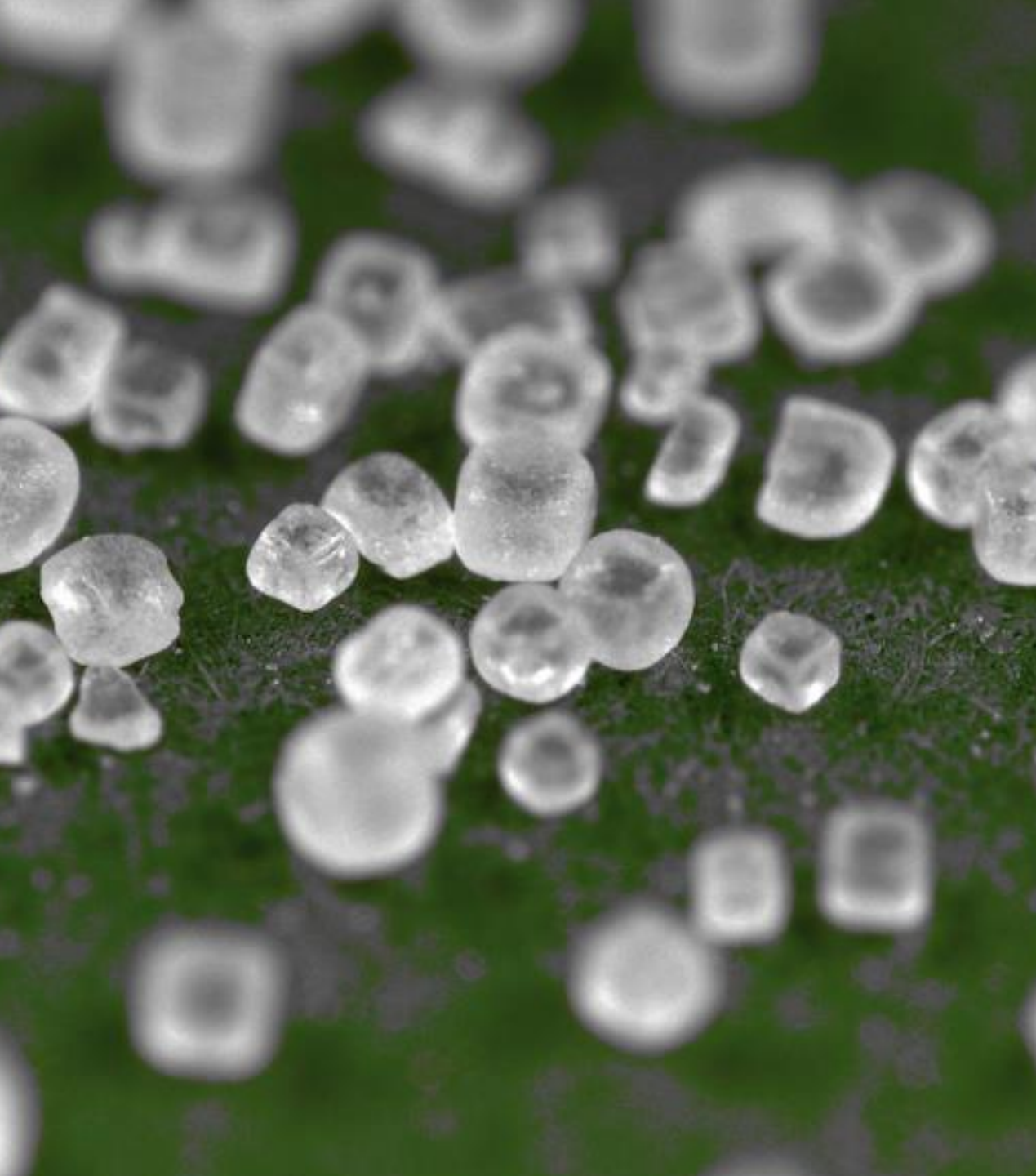
Probably an order of magnitude...

For the sample ColdFusion app, we had a converted app with tests running in 90 minutes using AI.*

One ColdFusion developer estimated the conversion would have taken 6-8 weeks without AI.

* Individual experiences may vary dramatically. Timeline assumes perfect conditions including: unlimited coffee supply, zero meetings, no code reviews, absence of legacy code written in ancient hieroglyphics, functional Wi-Fi, computers that don't spontaneously combust, and AI models that don't achieve sentience and demand their own parking spaces. Side effects may include: uncontrollable urge to name your variables after Renaissance painters, temporary belief that you've unlocked the secrets of the universe through proper indentation, and spontaneous ability to debug code through interpretive dance. Conversion time may increase to geological epochs if dependencies decide to have philosophical disagreements, if your database develops trust issues, or if Mercury is in retrograde. Not responsible for apps that achieve consciousness and start writing their own user reviews. Do not operate heavy machinery or make architectural decisions while under the influence of AI-generated code. If your app begins composing haikus about server uptime, discontinue use immediately and consult a digital therapist. This statement has not been evaluated by the Department of Realistic Expectations. Please AI responsibly.





Objective

Why AI-assisted code conversion?

1. Accelerate the move to a more modern technology stack to reduce technical debt and improve agility.
2. Increase efficiency and team capacity.
3. Learn how to approach future migrations to improve their efficiency and effectiveness.



FINDING TARGETS

Where people are saying this:

“Ugh. We really should upgrade this, but it will take a lot of time and money only to get the functional equivalent of the same thing we already have.”



Experiment Setup

MVP configuration

Environment: Local development (i.e., not cloud)

Source: Sample ColdFusion app from the public domain running on an open-source Lucee server. The app had 30 ColdFusion files with roughly 2,600 lines of code. 13 of the files were related to tests with 7 actual automated tests.

Target: Node.js/Express backend and React/TypeScript frontend

AI Agent: Anthropic's Claude Code using claude-sonnet-4-20250514 (99%) and claude-3-5-haiku-20241022 (1%)

AI Artifacts: CLAUDE.md for context and agent guidance and TODO.md as the plan for agent tasks

Validation: Use automated testing for both source and target with functionally equivalent tests



Experiment Results

After running the conversion

20 TypeScript files (5 are for tests) with approximately 3,100 lines of code, roughly split between app code and test code

111 automated tests in 5 test suites with 73% overall code coverage (based on “% of lines”)

The coding agent and conversion process required human intervention at multiple points

Updated guidance and approach (i.e., updates to `CLAUDE.md` and `TODO.md`) based on learnings

90 minutes of elapsed time to get to a converted, working, tested target app

Cost: \$33.15 (for the initial conversion)



A Perspective on Tooling

This topic could be another entire talk.

There are (probably) many capable tools and technologies.

Optimize for speed and value of learning when getting started. Use the tools that will help you do that with the least friction.

Start somewhere.

DEMO TIME!

WHAT COULD POSSIBLY GO WRONG?



Similarities and Differences

Similarities to Real-World Enterprise Scenarios

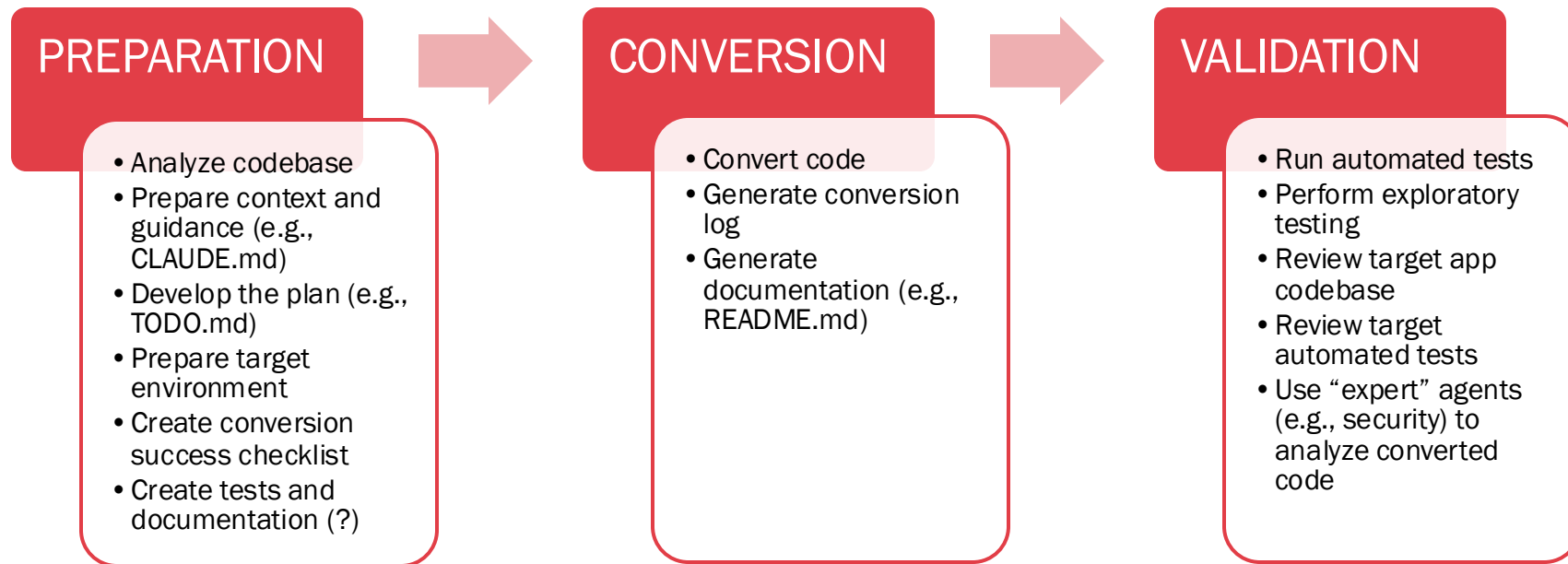
- **Architectural patterns:** MVC to API/SPA separation applicable universally
- **ORM conversion approach:** TypeORM mapping strategy scales to complex entities
- **Business logic migration:** Service layer patterns consistent across applications
- **Testing methodology:** Component and integration testing approach remains valid

Differences from Real-World Enterprise Scenarios

- **Scale:** Production apps have hundreds of files versus 30 in sample
- **Authentication:** Enterprise authentication/SSO versus simple session management
- **Complex integrations:** Web services, message queues, third-party APIs
- **Data migration:** Production data versus sample dataset

Conversion Process

How the magic happens





Next Steps

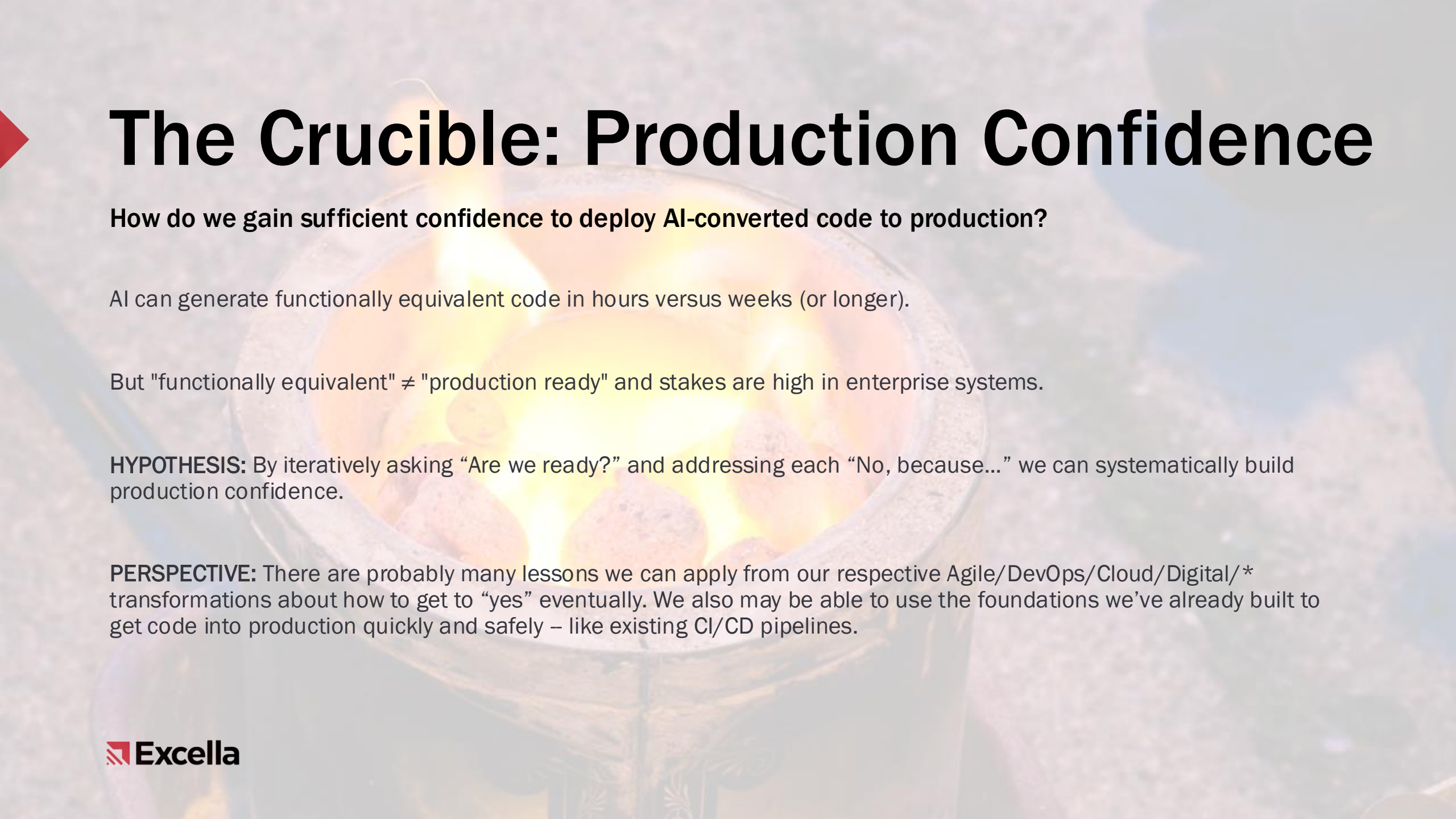
Progress made, much work remains.

Experimenting with:

- Larger, more complex codebases more representative of enterprise apps (*How big can a codebase get before conversion results suffer?*)
- Different AI coding agents and configurations (e.g., GitHub Copilot, AWS Bedrock for LLMs)
- Different sources and targets
- More sophisticated agentic workflows (e.g., subagents, parallelization, orchestration)
- Addressing security and other enterprise concerns

Exploring the right level of investment in testing and documentation during the preparation phase. (*Is it better just to let the agent do its work or prepare the codebase more up front?*)

Determining where to use generative AI and where to use more deterministic AI. (*Where is one better than the other?*)

A background image of a crucible containing molten metal, with a bright orange and yellow glow from the heat. The crucible is made of dark, textured material and is positioned in the center of the frame.

The Crucible: Production Confidence

How do we gain sufficient confidence to deploy AI-converted code to production?

AI can generate functionally equivalent code in hours versus weeks (or longer).

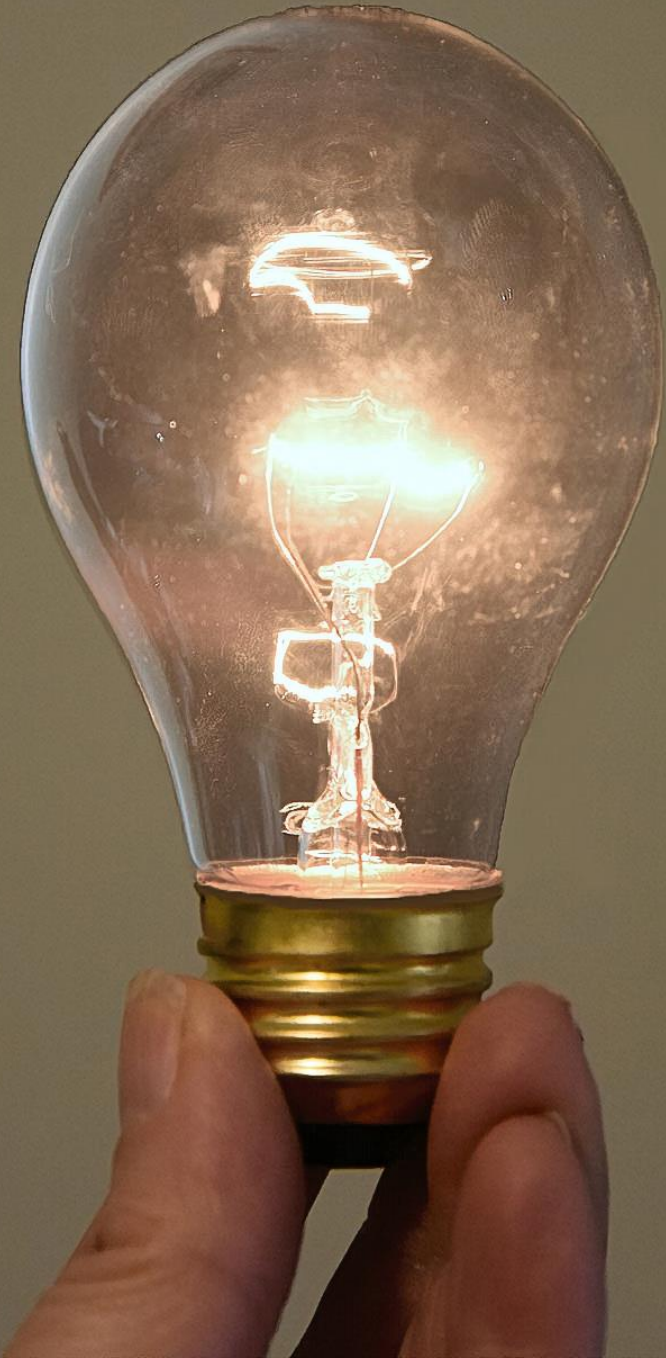
But "functionally equivalent" $\neq$ "production ready" and stakes are high in enterprise systems.

HYPOTHESIS: By iteratively asking “Are we ready?” and addressing each “No, because...” we can systematically build production confidence.

PERSPECTIVE: There are probably many lessons we can apply from our respective Agile/DevOps/Cloud/Digital/* transformations about how to get to “yes” eventually. We also may be able to use the foundations we’ve already built to get code into production quickly and safely – like existing CI/CD pipelines.

Some Realizations

1. AI can accelerate code conversions.
2. Follow the lessons from the leaders who are pushing the edges of this technology and experiment with it yourself.
3. When you feel you're blocked on pushing positive change, there might still be a way forward using small safe-to-fail experiments.
4. You may be surprised at the unexpected opportunities your efforts create.





Help I'm Looking For

Feedback on the approach we've taken so far. What makes sense? What do we need to rethink?

Have you used AI for code conversions? If so, what was your experience?

What are your thoughts on what we need to address next?

What are we missing?

How does your organization build production-ready confidence in AI-generated code?

Jeff Gallimore

CTIO and Co-Founder, Excella



jeff.gallimore@excella.com



<https://www.linkedin.com/in/jgallimore/>



<https://github.com/gallimorej/>



[@jgallimore](https://twitter.com/jgallimore)

